Runtime Lifecycle

Purpose

Every Runtime within Shio Runtime follows a predictable lifecycle.

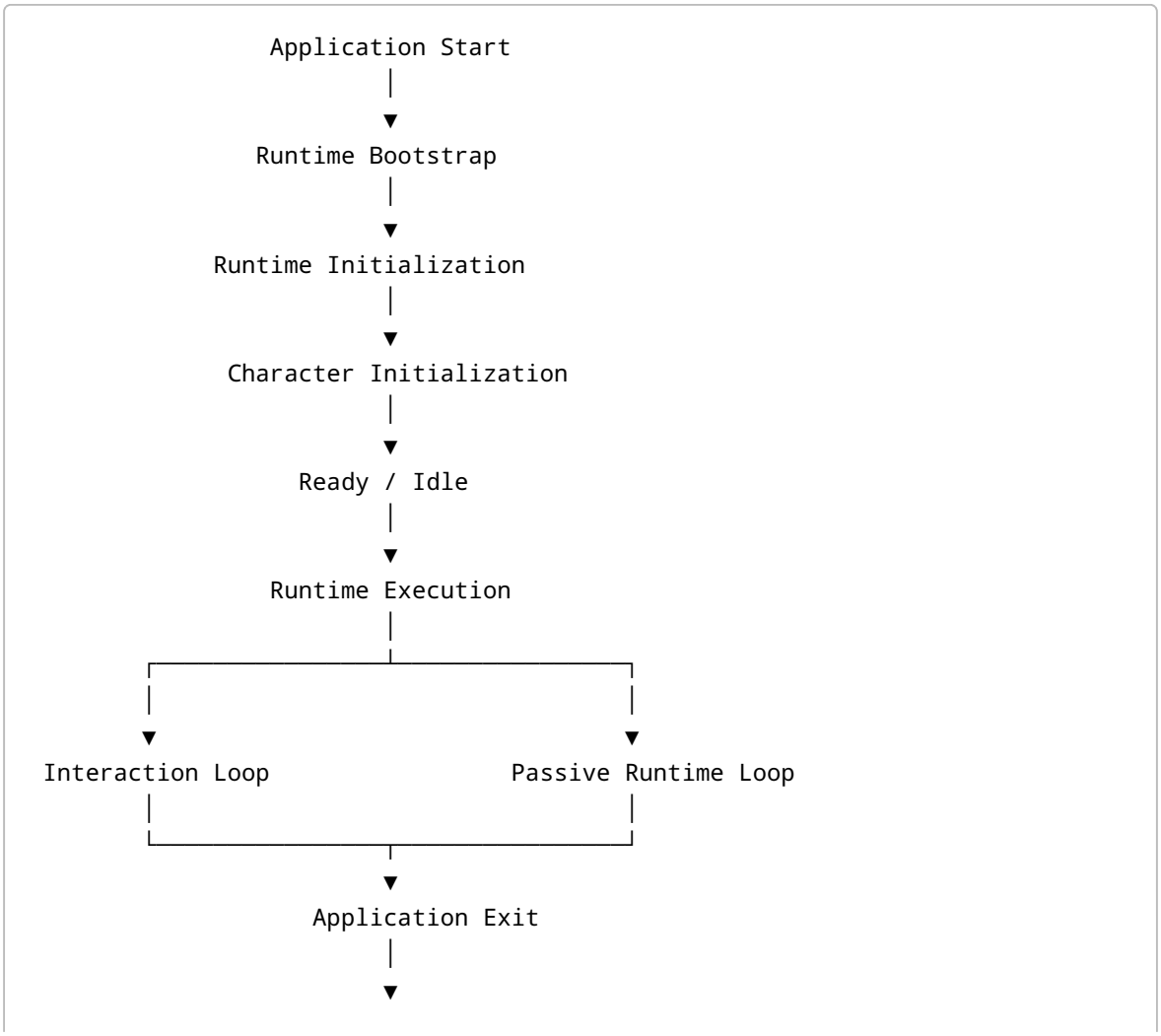
A consistent lifecycle allows Runtime systems to be initialized, started, paused, resumed and shut down without introducing special-case logic.

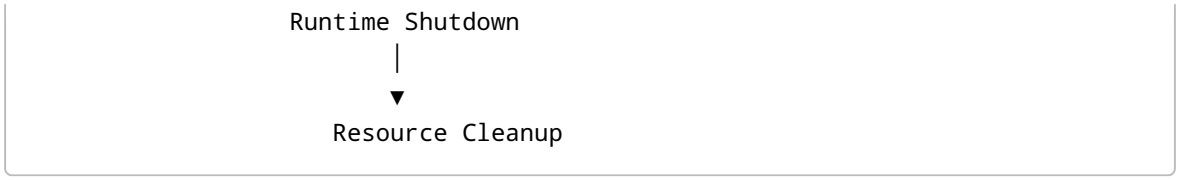
The lifecycle is independent of implementation details.

Whether a Runtime communicates with an LLM, renders a VRM model, synthesizes speech or stores memory is irrelevant.

Every Runtime participates in the same architectural lifecycle.

Lifecycle Overview





The Runtime lifecycle is intentionally linear.

Every Runtime enters and exits the system in a predictable way.

Phase 1 — Runtime Bootstrap

Bootstrap is responsible for creating Runtime instances.

At this stage, Runtime systems should not communicate with external services.

They should not load models.

They should not initialize graphics.

They should not start background tasks.

Bootstrap only creates the Runtime structure.

Its purpose is to assemble the architecture.

Phase 2 — Runtime Initialization

Initialization prepares each Runtime for execution.

Each Runtime initializes only its own responsibility.

Examples include:

Speech Runtime:

- initialize ASR;
- initialize TTS;
- prepare LLM client;
- create speech queues.

Avatar Runtime:

- initialize Three.js;
- load VRM;
- initialize animation controllers;

- initialize blink controller;
- initialize breathing controller.

Action Runtime:

- build Tool Registry;
- initialize Permission Guard;
- register available actions.

Memory Layer:

- open memory provider;
- restore cached data;
- prepare memory indexes.

Initialization is local.

A Runtime never initializes another Runtime.

Phase 3 — Character Initialization

Once every Runtime is initialized, the character itself is initialized.

This phase establishes the initial Character State.

Typical initial values include:

```
Mode        = Idle
Emotion     = Neutral
Attention   = None
Speech      = Silent
Activity    = Waiting
```

The character is now considered alive.

No interaction has occurred yet.

Phase 4 — Ready

The Runtime is now fully operational.

The character waits for external events.

Examples include:

- microphone input;

- keyboard shortcuts;
- external adapters;
- scheduled events;
- future memory events.

Ready represents an idle but fully operational system.

Phase 5 — Runtime Execution

Runtime Execution represents the normal operating state of the application.

This phase may continue indefinitely.

Runtime systems continuously observe:

- Character State;
- Runtime Events;
- external input.

They react independently while respecting Runtime boundaries.

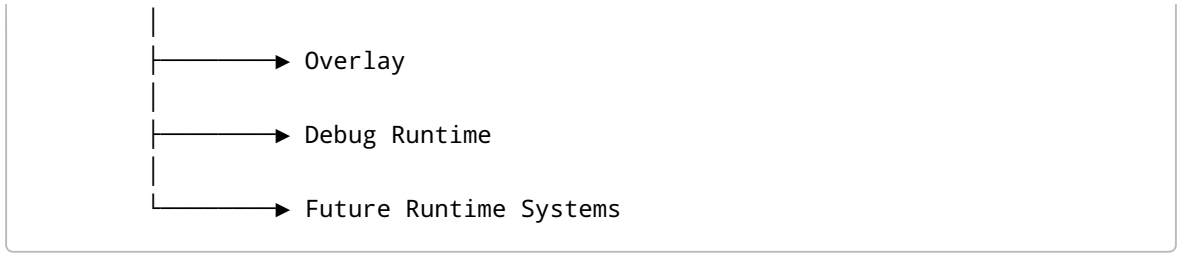
No Runtime becomes the central coordinator.

Behavior emerges through cooperation.

The Interaction Loop

A typical interaction follows the same architectural pattern.

```
External Input
        │
        ▼
Speech Runtime
        │
        ▼
Rule Gate
        │
        ▼
LLM
        │
        ▼
Character State Updated
        │
        ├──────► Avatar Runtime
        │
        ├──────► Experience Layer
```



No Runtime explicitly commands another Runtime to react.

Every Runtime independently determines whether the new Character State requires action.

Passive Runtime Execution

Not every Runtime requires external input.

Some Runtime systems continue operating continuously.

Examples include:

- blinking;
- procedural breathing;
- idle micro movements;
- animation blending;
- memory maintenance;
- scheduler timers.

These Runtime systems contribute to the illusion of life even while the character is silent.

Passive execution is considered a first-class architectural concept.

Runtime Suspension

Future versions of Shio Runtime may temporarily suspend selected Runtime systems.

Examples include:

- application minimized;
- overlay hidden;
- background execution;
- low power mode.

Suspension should pause work without destroying Runtime state.

Resuming should continue execution without requiring full initialization.

Phase 6 — Runtime Shutdown

Shutdown begins when the application is exiting.

Runtime systems stop accepting new work.

Current operations should complete whenever possible.

Examples include:

Speech Runtime:

- finish active speech generation;
- stop recognition;
- flush queues.

Avatar Runtime:

- stop rendering;
- stop animation loops;
- release graphics resources.

Memory Layer:

- persist pending data;
- flush caches;
- close storage.

Action Runtime:

- complete active actions;
- reject new actions.

Shutdown should be graceful.

Unexpected termination should be treated as an exceptional condition rather than the normal execution path.

Resource Cleanup

After shutdown, every Runtime releases resources that it owns.

Examples include:

- GPU resources;
- audio devices;
- network connections;
- timers;

- worker threads;
- cached objects.

A Runtime must never release resources owned by another Runtime.

Ownership and cleanup responsibilities always remain aligned.

Runtime Lifecycle Contract

Every Runtime should expose a consistent lifecycle.

The exact implementation may differ, but the conceptual contract remains the same.

```
Bootstrap

↓

Initialize

↓

Start

↓

Observe

↓

React

↓

Pause (future)

↓

Resume (future)

↓

Shutdown

↓

Dispose
```

This contract provides architectural consistency across every Runtime in the system.

Future Runtime systems should naturally integrate into this lifecycle without requiring special handling.

Architectural Guarantees

Following the Runtime Lifecycle provides several guarantees:

- predictable startup behavior;
- deterministic shutdown;
- clear ownership of resources;
- independent Runtime initialization;
- independent Runtime evolution;
- reduced coupling;
- simplified debugging;
- future plugin compatibility.

A Runtime should always be understandable by observing only its own lifecycle and responsibilities.

No Runtime should depend on hidden initialization order or undocumented side effects.

This consistency is one of the key architectural foundations of Shio Runtime.